

GIẢI THUẬT KÝ SỐ PDF

Dự án eGuarantee • SHA256withRSA • CMS/PKCS#7 detached

Tài liệu mô tả triển khai hiện tại trong **PdfSignatureService.java** và luồng điều phối của **GuaranteeService.java**. Đối chiếu mã nguồn ngày 01/10/2026.

Đầu vào: stream PDF gốc, private key và chuỗi chứng thư trong keystore.

Đầu ra: stream PDF chứa chữ ký CMS và chứng thư. Private key không được đưa vào PDF.

1. Luồng nghiệp vụ tổng thể

Bước 1. Kiểm tra quyền và hồ sơ

Người dùng phải có role CHECKER; hồ sơ tồn tại, có trạng thái APPROVED và chưa SIGNED. Nếu không đạt, dừng và trả lỗi nghiệp vụ.

Bước 2. Đánh dấu chờ ký

Ghi PENDING_SIGN, người cập nhật, thời gian cập nhật và lịch sử SIGN. Sau đó gọi tác vụ nền bằng CompletableFuture.runAsync().

Bước 3. Tìm và tải tài liệu

Tìm GuaranteeFile theo publicId. Code kiểm tra file thuộc hồ sơ khi guaranteeld của file khác null. Xác định PDF/DOCX và tải file gốc từ Cloudinary.

Bước 4. Ký PDF

Với định dạng PDF, gọi signPdf(in, out). Các trang tiếp theo mô tả chi tiết giải thuật này.

Bước 5. Upload và cập nhật

Upload file đã ký; cập nhật URL, publicId, kích thước và isDigitallySigned. Sau khi hoàn tất, ghi SIGNED và lịch sử thành công. Nếu tác vụ phát sinh exception, ghi FAILED khi tìm được hồ sơ.

CHECKER + APPROVED + chưa SIGNED

→ PENDING_SIGN

→ tải file → ký PDF → upload → cập nhật dữ liệu

→ SIGNED (thành công) / FAILED (có exception)

Tác vụ chạy ở nền nên phản hồi ban đầu có thể vẫn là PENDING_SIGN. Trạng thái SIGNED được đặt sau quá trình ký, upload và cập nhật; code chưa tự xác thực mật mã chữ ký sau khi tạo.

2. Nạp khóa và mở tài liệu

Điểm vào: `signPdf(InputStream inputPdf, OutputStream outputPdf)`, gọi `initKeyStore()` trước khi mở PDF.

Bước 6. Khởi tạo keystore

Nếu `privateKey` đã có trong bộ nhớ thì bỏ qua việc nạp lại. Nếu chưa có, tạo `KeyStore` với định dạng PKCS12 và mở resource `classpath:keystore.p12`.

Bước 7. Lấy khóa và chứng thư

Dùng mật khẩu cấu hình trong service để mở keystore, lấy private key và certificate chain theo alias `eguarantee`. Nếu private key bằng null thì báo lỗi. Code hiện tại chưa kiểm tra riêng trường hợp certificate chain thiếu hoặc rỗng.

Thành phần	Vai trò
<code>keystore.p12</code>	Lưu private key và chứng thư.
Mật khẩu keystore	Cho phép đọc keystore và lấy khóa; không phải khóa dùng để ký.
<code>privateKey</code>	Khóa RSA bí mật dùng để tạo chữ ký.
<code>certificateChain[0]</code>	Chứng thư người ký, có public key tương ứng.
<code>certificateChain</code>	Chuỗi chứng thư đính kèm vào CMS.
<code>SECRET_KEY</code>	Dùng cho JWT; không tham gia ký tài liệu.

Bước 8. Mở PDF bằng PDFBox 3

`RandomAccessReadBuffer` sao chép `inputPdf` vào buffer trong bộ nhớ. `Loader.loadPDF(source)` phân tích cấu trúc PDF và trả về `PDDocument`. `try-with-resources` đóng document và source khi kết thúc.

```
initKeyStore();
try (RandomAccessReadBuffer source =
    new RandomAccessReadBuffer(inputPdf);
    PDDocument document = Loader.loadPDF(source)) {
    // Khai báo chữ ký và lưu PDF
}
```

Tham chiếu mã: `PdfSignatureService.java`, `initKeyStore()` dòng 44; `signPdf()` dòng 65–69.

3. Chuẩn bị chữ ký và vùng byte cần ký

Bước 9. Khai báo PDSignature

Đặt Filter là Adobe.PPKLite; SubFilter là adbe.pkcs7.detached; tên eGuarantee System; địa điểm Hanoi, Vietnam; lý do phê duyệt thư bảo lãnh; thời gian là thời gian hiện tại của server. Các trường này mô tả chữ ký; chứng thư mới gắn public key với danh tính người ký.

Bước 10. Đăng ký callback

document.addSignature(signature, this) đăng ký đối tượng chữ ký và hàm xử lý của service. this triển khai SignatureInterface; PDFBox sẽ gọi sign(content) khi lưu tài liệu.

Bước 11. Bắt đầu lưu incremental

document.saveIncremental(outputPdf) chuẩn bị phần cập nhật PDF, dành vùng /Contents cho chữ ký và thiết lập /ByteRange. PDFBox cung cấp stream các vùng byte cần ký cho callback.

```
signPdf()
→ document.addSignature(signature, this)
→ document.saveIncremental(outputPdf)
→ PDFBox chuẩn bị ByteRange
→ gọi this.sign(content)
→ nhận byte[] CMS và nhúng vào PDF
→ hoàn tất outputPdf
```

ByteRange hoạt động như thế nào? Chữ ký bao phủ các vùng byte được chỉ định, bỏ qua vùng /Contents chứa chính chữ ký. Đây là cách tránh việc chữ ký phải bao gồm chính các byte của nó.

```
PDF: [ vùng A ][ /Contents: chữ ký CMS ][ vùng B ]
ByteRange minh họa: [0, a, b, c]
Dữ liệu ký D = PDF[0:a] nối với PDF[b:b+c]
```

Trong ví dụ, a là vị trí bắt đầu khoảng trống, b là vị trí ngay sau khoảng trống và c là độ dài phần sau. Các số này do PDFBox tính; service không tự tính hoặc gán ByteRange.

content truyền vào sign() là dữ liệu PDFBox chuẩn bị để ký, không phải đơn thuần stream PDF gốc. Metadata chữ ký và cấu trúc cập nhật nằm trong các vùng được ký, trừ vùng chứa chữ ký.

Tham chiếu mã: PdfSignatureService.java, dòng 71–83.

4. Tạo chữ ký CMS bằng SHA256withRSA

Bước 12. Chuẩn bị bộ ký

JcaContentSignerBuilder("SHA256withRSA").build(privateKey) chọn SHA-256 và RSA, gắn private key. Đây là bước cấu hình; chữ ký hoàn chỉnh được tạo trong generate().

Bước 13. Chuẩn bị bộ sinh CMS

Tạo CMSSignedDataGenerator; thêm SignerInfoGenerator kết hợp bộ tính digest, ContentSigner và certificateChain[0]; thêm toàn bộ certificate chain vào gói CMS.

Bước 14. Đọc dữ liệu cần ký

Lặp đọc content bằng buffer 8192 byte, ghi vào ByteArrayOutputStream, sau đó đóng gói thành CMSProcessableByteArray. Vòng lặp chỉ sao chép dữ liệu; chưa trực tiếp băm. 8192 là kích thước mỗi lần đọc, không phải kích thước chữ ký.

```
while ((read = content.read(buffer)) != -1) {
    baos.write(buffer, 0, read);
}
CMSProcessableByteArray msg =
    new CMSProcessableByteArray(baos.toByteArray());
CMSSignedData signedData = gen.generate(msg, false);
return signedData.getEncoded();
```

Bước 15. Băm, ký và đóng gói

Bouncy Castle tính digest nội dung PDF. Với cấu hình CMS mặc định, digest được đưa vào thuộc tính messageDigest; tập thuộc tính đã ký được mã hóa và ký bằng SHA256withRSA. CMS chứa thuật toán, thuộc tính, chữ ký và chứng thư.

```
D = các byte PDF được ByteRange chỉ định
H = SHA256(D) // 32 byte
A = thuộc tính CMS, có messageDigest = H
S = RSA-SHA256-Sign(privateKey, DER(A))
CMS = đóng gói A, S, thuật toán và chứng thư
```

Ý nghĩa false: generate(msg, false) tạo CMS detached, không nhúng thêm bản sao nội dung PDF vào CMS. Gói CMS vẫn được nhúng trong PDF. Private key không nằm trong CMS. Ký số không mã hóa nội dung PDF để giữ bí mật.

Tham chiếu mã: PdfSignatureService.java, dòng 92–119. Các chi tiết băm và ký được thư viện thực hiện bên trong generate().

5. Hoàn tất PDF và xác thực chữ ký

Bước 16. Nhúng CMS vào tài liệu

Callback trả byte[] của CMS cho PDFBox. PDFBox điền chữ ký vào vùng /Contents đã dành sẵn và hoàn tất outputPdf. saveIncremental() giữ dữ liệu gốc và thêm phần cập nhật ở cuối PDF.

File kết quả chứa chữ ký mật mã và chứng thư. Service chưa thiết lập hình ảnh hoặc vùng hiển thị chữ ký trên trang. Khối chữ ký CMS là đối tượng nhị phân, khác với ảnh chữ ký hoặc con dấu.

Giải thuật xác thực ở bên nhận (mô tả nguyên lý; service hiện tại không triển khai bước này):

1. Mở PDF; đọc ByteRange và CMS trong /Contents.
2. Ghép các vùng byte theo ByteRange thành D_mới.
3. Tính $H_{\text{mới}} = \text{SHA256}(D_{\text{mới}})$.
4. Đối chiếu $H_{\text{mới}}$ với messageDigest trong CMS.
5. Lấy public key từ chứng thư người ký.
6. Xác thực chữ ký RSA trên các thuộc tính CMS.
7. Kiểm tra chuỗi chứng thư, thời hạn và thu hồi.
8. Đánh giá cập nhật PDF sau bản đã ký, nếu có.

Toàn vẹn: Nếu các byte thuộc vùng đã ký thay đổi, digest không còn khớp hoặc chữ ký không xác thực được. Một bản cập nhật nối thêm sau khi ký có thể giữ nguyên chữ ký của bản trước; trình đọc cần đánh giá riêng phần thay đổi mới.

Tin cậy: Chữ ký xác thực về mặt mật mã chứng minh sự tương ứng giữa dữ liệu và khóa ký. Để tin cậy danh tính người ký còn cần đánh giá chứng thư và chuỗi CA; việc đính kèm certificate chain không tự tạo ra sự tin cậy.

Thời gian: setSignDate(Calendar.getInstance()) lấy giờ server. Đây không phải dấu thời gian được dịch vụ TSA độc lập chứng thực.

Xử lý lỗi: Exception trong sign() được ghi log và bọc thành IOException; các lỗi trong signPdf() được truyền cho nơi gọi. Tác vụ nghiệp vụ bắt exception và chuyển hồ sơ sang FAILED khi có thể tìm được hồ sơ.

6. Mã giả tổng hợp và đặc điểm triển khai

THỦ TỤC KÝ_PDF(inputPdf, outputPdf):

Nếu chưa nạp khóa:

Mở keystore PKCS12

Lấy privateKey và certificateChain theo alias

Nếu privateKey không tồn tại: báo lỗi

Mở PDF bằng Loader.loadPDF(buffer(inputPdf))

Tạo metadata chữ ký PKCS#7 detached

Đăng ký callback KÝ_CMS

Lưu incremental:

PDFBox tạo ByteRange và stream dữ liệu ký

Gọi KÝ_CMS, nhận CMS và nhúng vào /Contents

Đóng tài liệu và buffer

HÀM KÝ_CMS(content):

Cấu hình SHA256withRSA với privateKey

Thêm thông tin người ký và chuỗi chứng thư

Đọc content thành mảng byte D

Sinh CMS detached cho D bằng thư viện

Trả về các byte CMS đã mã hóa

Nếu lỗi: ghi log và ném IOException

Khóa hệ thống dùng chung: Các CHECKER được phép ký đều dùng khóa alias eguarantee trong cùng keystore. Username được ghi trong lịch sử nghiệp vụ; service chưa chọn khóa riêng theo tài khoản.

Trạng thái và xác thực: SignatureStatus chỉ phản ánh tiến trình nghiệp vụ. Việc ghi SIGNED không bao gồm bước xác thực chữ ký sau tạo. Service chưa tích hợp TSA, kiểm tra tin cậy chứng thư hay thu hồi chứng thư.

Bộ nhớ: PDF được đọc vào RandomAccessReadBuffer; callback cũng sao chép toàn bộ dữ liệu ký vào mảng byte. Lượng bộ nhớ cần dùng tăng theo kích thước tài liệu.

Mã nguồn tham chiếu: src/main/java/com/example/ecommerce/service/PdfSignatureService.java; GuaranteeService.java, phần điều phối ký dòng 260–378. Số dòng tương ứng mã nguồn tại thời điểm xuất tài liệu.

Tài liệu tham khảo

[PDFBox 3: mở PDF bằng Loader](#)

[Apache PDFBox: ví dụ ký số](#)

[Bouncy Castle: sinh chữ ký CMS detached](#)

[Bouncy Castle: thuộc tính CMS mặc định](#)